

Database Practicum

WEEK 5 ASSIGNMENT



Farrel Tsaqif Anindyo
24/536735/PA/22769

I. Student Activity

1. I am using the previous student tables from the previous exercise because it has the same data.
2. Tampilkan kolom city address tanpa ada duplikat.

```
MariaDB [academic]> select Name
-> from student
-> WHERE Name like 'F%';
+-----+
| Name |
+-----+
| Feni |
| Farah |
| Fandi |
+-----+
3 rows in set (0.001 sec)
```

3. Tampilkan nilai IPK Maksimum IPK dari mahasiswa dari Jakarta.

```
MariaDB [academic]> select DISTINCT cityAddress
-> from student;
+-----+
| cityAddress |
+-----+
| Jakarta |
| Yogyakarta |
| Surabaya |
| Banjarmasin |
| Medan |
+-----+
5 rows in set (0.042 sec)
```

4. Tampilkan nilai IPK Minimum dari mahasiswa departemen Komputer.

```
MariaDB [academic]> select Name, MAX(IPK)
-> from student
-> WHERE cityAddress = 'Jakarta';
```

Name	MAX(IPK)
Adi	3.80

```
1 row in set (0.001 sec)
```

5. Tampilkan nilai IPK Rata-rata dari mahasiswa departemen Matematika.

```
MariaDB [academic]> select Department, AVG(IPK)
-> from student
-> WHERE Department = 'Math';
```

Department	AVG(IPK)
Math	2.725000

```
1 row in set (0.001 sec)
```

6. Tampilkan jumlah mahasiswa dari departemen Komputer.

```
MariaDB [academic]> select Department, COUNT(Name)
-> from student
-> Where Department = 'Computer'
-> GROUP BY Department;
```

Department	COUNT(Name)
Computer	7

```
1 row in set (0.001 sec)
```

7. Tampilkan jumlah mahasiswa dari setiap departemen.

```
MariaDB [academic]> select Department, COUNT(Name)
-> from student
-> GROUP BY Department;
```

Department	COUNT(Name)
NULL	4
Computer	7
Math	4
Null	1
Physics	4

5 rows in set (0.001 sec)

8. Tampilkan jumlah mahasiswa yang dikelompokkan berdasarkan city address dengan usia dibawah 20 tahun.

```
MariaDB [academic]> select cityAddress, COUNT(Name)
-> from student
-> WHERE Age < 20
-> GROUP BY cityAddress;
```

cityAddress	COUNT(Name)
Banjarmasin	1
Jakarta	4
Medan	1
Surabaya	4
Yogyakarta	4

5 rows in set (0.001 sec)

II. Questions

1. Mengapa kita membutuhkan agregasi DML?

Aggregate functions are used to make a calculation to the data using functions like MAX, MIN, AVG, SUM, etc. It is useful because you can simplify the data into meaningful information especially in the bigger dataset. For example if you want to know the maximum value in some column you can use a MAX function. So

you don't need to do a lot of calculations and searching and it will save a lot of time.

2. Apa pentingnya pengelompokan data?

Data grouping in Data Manipulation Language (DML) is important because it organizes and summarizes large datasets, making them easier to analyze and interpret by creating summary information like totals and averages for specific categories. It highlights patterns, provides actionable insights, and allows for more efficient estimation by reducing complexity. Essentially, DML grouping transforms raw data into meaningful, organized information that facilitates decision-making and a deeper understanding of the data.

3. Apa perbedaan antara WHERE dan HAVING?

WHERE is only applied to individual rows before they are grouped and aggregated (if GROUP BY exists) and it can't contain any aggregate functions.

HAVING can be applied to a group of rows and it is formed after the GROUP BY clause and any aggregated functions have been calculated and it can also contain aggregate functions.

III. Exercise

1. MAX

Tampilkan nilai kredit terbesar (creditLimit) dari seluruh customer.

```
mysql> select MAX(creditLimit)
-> from customers;
+-----+
| MAX(creditLimit) |
+-----+
|          227600.00 |
+-----+
1 row in set (0.00 sec)
```

2. MIN

Tampilkan tanggal order paling awal yang tercatat di tabel orders.

```
mysql> select MIN(orderDate)
-> from orders;
+-----+
| MIN(orderDate) |
+-----+
| 2003-01-06      |
+-----+
1 row in set (0.00 sec)
```

3. AVG

Hitung rata-rata harga beli (buyPrice) produk dalam kategori Classic Cars.

```
mysql> select AVG(buyPrice)
-> from products
-> WHERE productLine = 'Classic Cars';
+-----+
| AVG(buyPrice) |
+-----+
| 64.446316      |
+-----+
1 row in set (0.01 sec)
```

4. SUM

Hitung total penjualan ($\text{quantityOrdered} \times \text{priceEach}$) dari tabel orderdetails.

```
mysql> select SUM(quantityOrdered * priceEach)
-> from orderdetails;
+-----+
| SUM(quantityOrdered * priceEach) |
+-----+
| 9604190.61                        |
+-----+
1 row in set (0.00 sec)
```

5. COUNT

Hitung jumlah customer dari negara USA.

```
mysql> select COUNT(customerName)
-> from customers
-> WHERE country = 'USA';
+-----+
| COUNT(customerName) |
+-----+
|                    36 |
+-----+
1 row in set (0.00 sec)
```

6. GROUP BY

Tampilkan jumlah customer per negara (country).

```
mysql> select country, COUNT(customerName)
-> from customers
-> GROUP BY country;
```

country	COUNT(customerName)
Australia	5
Austria	2
Belgium	2
Canada	3
Denmark	2
Finland	3
France	12
Germany	13
Hong Kong	1
Ireland	2
Israel	1
Italy	4
Japan	2
Netherlands	1
New Zealand	4
Norway	3
Philippines	1
Poland	1
Portugal	2
Russia	1
Singapore	3
South Africa	1
Spain	7
Sweden	2
Switzerland	3
UK	5
USA	36

27 rows in set (0.00 sec)

7. GROUP BY + SUM

Hitung total nilai penjualan untuk setiap produk (group berdasarkan productCode).


```
mysql> select productCode, SUM(buyPrice)
-> from products
-> GROUP BY productCode;
```

productCode	SUM(buyPrice)
S10_1678	48.81
S10_1949	98.58
S10_2016	68.99
S10_4698	91.02
S10_4757	85.68
S10_4962	103.42
S12_1099	95.34
S12_1108	95.59
S12_1666	77.90
S12_2823	66.27
S12_3148	89.14
S12_3380	75.16
S12_3891	83.05
S12_3990	31.92
S12_4473	55.70
S12_4675	58.73
S18_1097	58.33
S18_1129	83.51
S18_1342	60.62
S18_1367	24.26
S18_1589	65.96
S18_1662	77.27
S18_1749	86.70
S18_1889	53.90
S18_1984	93.89
S18_2238	101.51
S18_2248	33.30
S18_2319	74.86
S18_2325	58.48
S18_2432	24.92
S18_2581	49.00
S18_2625	24.23

8. GROUP BY + AVG + HAVING

Tampilkan daftar productLine yang memiliki rata-rata harga jual (MSRP) di atas 80.

```
mysql> select productLine, AVG(MSRP)
-> FROM products
-> GROUP BY productLINE
-> HAVING AVG(MSRP) > 80;
```

productLine	AVG(MSRP)
Classic Cars	118.021053
Motorcycles	97.178462
Planes	89.515833
Ships	86.563333
Trucks and Buses	103.183636
Vintage Cars	87.095833

6 rows in set (0.00 sec)

9. GROUP BY + COUNT + HAVING

Tampilkan daftar negara yang memiliki lebih dari 10 customer.

```
mysql> select country, COUNT(customerNumber)
-> from customers
-> GROUP BY country
-> HAVING COUNT(customerNumber) > 10;
```

country	COUNT(customerNumber)
France	12
Germany	13
USA	36

3 rows in set (0.00 sec)

10. GROUP BY + SUM + HAVING

Hitung total penjualan per orderNumber, lalu tampilkan hanya order dengan total penjualan di atas 50.000.

```
mysql> select orderNumber, SUM(priceEach)
-> from orderdetails
-> GROUP BY orderNumber
-> HAVING SUM(priceEach) > 50000;
```

Empty set (0.00 sec)